Лабораторная работа №4
Зусь Александра, гр. 4404

Задание: Используя функциональную декомпозицию перепишите задачу 5 для вычисления значения по формуле:

$$\frac{\sum x + \sum y + \sum z}{3N}$$

где N – количество точек. Используйте глобальную сумму и критические секции.

Код программы:

```
#include <iostream>#include <fstream>#include <vector>#include <string>#include <chrono>#include <omp.h>#include
<cstdlib>#include <ctime>#include <iomanip>
using namespace std;
struct Point {
    double x, y, z;
};
// ==================== Генерация файла ====================void generate_points(const string& filename, int
count) {
    ofstream out(filename);
    if (!out) {
        cerr << "Cannot create file " << filename << endl;
        return;
    }

    srand(time(nullptr));

    for (int i = 0; i < count; ++i) {
        double x = (rand() % 20000 - 10000) / 100.0;
        double y = (rand() % 20000 - 10000) / 100.0;
        double z = (rand() % 20000 - 10000) / 100.0;
        out << x << " " << y << " " << z << "\n";
    }

    cout << "File \"" << filename << "\" created with " << count << " points.\n";
}
// ==================== Чтение точек ====================bool read_points(const string& filename,
vector<Point>& points) {
    ifstream in(filename);
    if (!in) return false;

    double x, y, z;
    while (in >> x >> y >> z) {
        points.push_back({ x, y, z });
    }
    return !points.empty();
}
// ==================== Вычисление по формуле с critical ====================double compute_formula(const
vector<Point>& points) {
    double global_sum = 0.0;
    int n = (int)points.size();
#pragma omp parallel
    {
        double local_sx = 0.0;
        double local_sy = 0.0;
        double local_sz = 0.0;
#pragma omp sections
        {#pragma omp section
            {for (int i = 0; i < n; ++i) local_sx += points[i].x;}#pragma omp section
            {for (int i = 0; i < n; ++i) local_sy += points[i].y;}#pragma omp section
            {for (int i = 0; i < n; ++i) local_sz += points[i].z;}
        }

        // Безопасное добавление в глобальную сумму#pragma omp critical
        {global_sum += local_sx + local_sy + local_sz;}
```

```
    }

    return global_sum / (3.0 * n);
}
// ==================== main ====================int main(int argc, char* argv[]) {
    // Режим генерации файла
    if (argc >= 2 && string(argv[1]) == "generate") {
        int count = (argc >= 3) ? atoi(argv[2]) : 1000000;
        generate_points("points.txt", count);
        return 0;
    }

    if (argc != 3) {
        cerr << "Usage:\n";
        cerr << "  Generate:  " << argv[0] << " generate [count]\n";
        cerr << "  Calculate: " << argv[0] << " <points_file> <num_threads>\n";
        return 1;
    }

    string filename = argv[1];
    int threads = atoi(argv[2]);
    omp_set_num_threads(threads);

    vector<Point> points;
    if (!read_points(filename, points)) {
        cerr << "Cannot read points from " << filename << endl;
        return 1;
    }

    cout << "Points loaded: " << points.size() << endl;
    cout << "Threads: " << threads << endl << endl;

    auto start = chrono::high_resolution_clock::now();
    double result = compute_formula(points);
    auto end = chrono::high_resolution_clock::now();
    double time = chrono::duration<double>(end - start).count();

    cout << fixed << setprecision(8);
    cout << "Result = " << result << endl;
    cout << "Time   = " << time << " s" << endl;

    return 0;
}
```

```
D:\Code\4laba\sum_formula\x64\Release>sum_formula.exe generate 1000000
File "points.txt" created with 1000000 points.
```

Рисунок 1. Работоспособность функции генерации файла с точками.

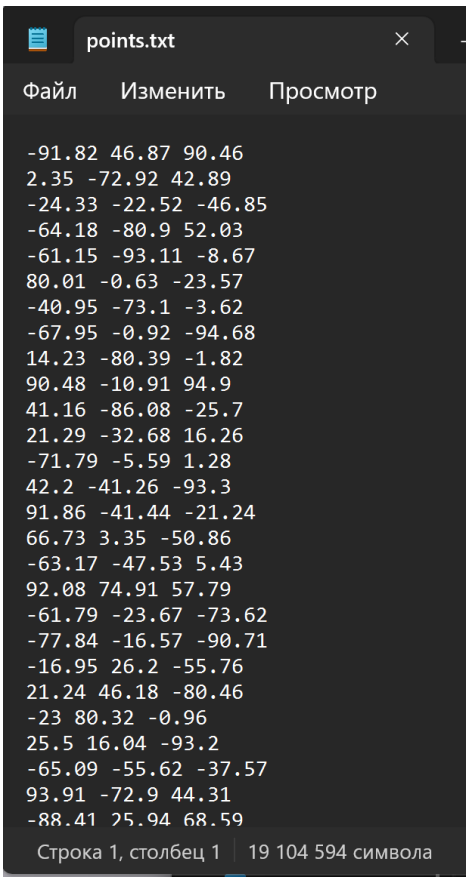


Рисунок 2. Содержание сгенерированного файла «points.txt».

```
D:\Code\4laba\sum_formula\x64\Release>sum_formula.exe points.txt 1
Points loaded: 1000000
Threads: 1

Result = -14.09655618
Time   = 0.00311340 s
```

Рисунок 3. Первый запуск: число потоков = 1, число точек = 1 000 000.

```
D:\Code\4laba\sum_formula\x64\Release>sum_formula.exe points.txt 2
Points loaded: 1000000
Threads: 2

Result = -14.09655618
Time   = 0.00345280 s
```

Рисунок 4. Второй запуск: число потоков = 2, число точек = 1 000 000.

```
D:\Code\4laba\sum_formula\x64\Release>sum_formula.exe points.txt 3
Points loaded: 1000000
Threads: 3

Result = -14.09655618
Time   = 0.00406390 s
```

Рисунок 5. Третий запуск: число потоков = 3, число точек = 1 000 000.

```
D:\Code\4laba\sum_formula\x64\Release>sum_formula.exe points.txt 1
Points loaded: 10000000
Threads: 1

Result = -14.09956423
Time   = 0.03049590 s

D:\Code\4laba\sum_formula\x64\Release>sum_formula.exe points.txt 2
Points loaded: 10000000
Threads: 2

Result = -14.09956423
Time   = 0.02992910 s

D:\Code\4laba\sum_formula\x64\Release>sum_formula.exe points.txt 3
Points loaded: 10000000
Threads: 3

Result = -14.09956423
Time   = 0.02859970 s
```

Рис 6. Четвёртый, пятый, шестой запуски: число потоков = 1-3, число точек = 10 000 000.

Анализ результатов

Эксперименты проводились на файле, содержащем 1 000 000 и 10 000 000 точек.

Сравнение времени выполнения:

Число потоков	Время выполнения	Число точек
1	0.00311 с	1 000 000
2	0.00345 с	1 000 000
3	0.00406 с	1 000 000
1	0.03049 с	10 000 000
2	0.02999 с	10 000 000
3	0.02859 с	10 000 000

При 1 000 000 точек

Время выполнения почти не уменьшается при увеличении числа потоков, а на 2 и 3 потоках даже немного растёт. Это связано с тем, что объём работы небольшой, и накладные расходы на создание потоков становятся заметными.

При 10 000 000 точек

Ситуация немного лучше. При переходе с 1 на 2 и 3 потока время постепенно уменьшается (с 0.0305 с до 0.0286 с). То есть при большем объёме данных небольшой выигрыш от параллельной работы уже появляется.

Вывод: В программе используется функциональная декомпозиция с тремя секциями. Поэтому даже при увеличении числа потоков сильного ускорения нет. Однако при росте объёма данных (с 1 млн до 10 млн точек) небольшое ускорение на 2-3 потоках становится более заметным. Это подтверждает, что при очень маленьком объёме работы накладные расходы на потоки могут полностью перекрывать пользу от распараллеливания.